ws-30

AGENDA is to …

TOTAL RECALL

Build Tools. Maven

Maven Installation and Configuration

- Windows installation requires setting the PATH variable after unpacking.
- Project-specific configuration is stored in the pom.xml file.
- Global tool configuration is located in the settings.xml file.
- Local repository storage is placed inside the user's .m2 folder.
- Official materials and guidelines reside on the Apache Maven website.

Standard Project Structure

- Maven enforces a strict, predictable structural directory layout.
- Application source code is placed inside src/main/java.
- Main application resources belong in src/main/resources.
- Test source files are kept inside src/test/java.
- Test resource files are saved within src/test/resources.
- Compiled binaries and target packages are outputted to the target/ directory.

Build Lifecycle and Terminology

- Lifecycle: Overall sequence of steps containing default, clean, and site options.
- Phase: An official milestone or step within a build lifecycle.
- Goal: A specific unit of work provided by a plugin.
- The default build lifecycle consists of 23 phases.
- The initial phase executed in the default lifecycle is validate.

Core Commands and Plugins

- mvn clean removes previously generated build files from the system.
- mvn compile compiles production code via the core Compiler plugin.
- mvn test triggers unit test execution using the Surefire plugin.
- mvn package packages compiled binaries into a JAR or WAR archive.
- mvn install deploys the final package into the local repository.
- Plugin definitions can be queried using the mvn help:describe utility.

Dependency Management

- Library requirements are stated inside the core configuration file.
- Artifact searches are performed using Maven Central or MVNRepository.
- Dependency scopes dictate the precise lifecycle phases an artifact covers.
- Supported scopes contain compile, test, provided, runtime, and system.
- The compile scope acts as the standard default fallback choice.
- The test scope limits artifact presence to test tasks alone.

Maven Project Structure

- src/ directory for all project source files
- target/ folder storing compiled build outputs
- pom.xml root file for project configuration
- src/main/java/ containing core application logic
- src/main/resources/ holding main configuration files
- src/test/java/ housing test code files
- src/test/resources/ keeping test-specific resource asset

Maven Build Lifecycle and Stage

- Sequential execution of defined lifecycle phases
- clean stage to delete previous build outputs
- compile stage to compile application source code
- package stage to bundle code into binaries
- verify stage where integration tests run

Maven Dependency Management

- Automatic dependency resolution from remote repositories
- compile set as the default dependency scope
- alternative scopes including provided, runtime, and test
- <exclusions> tag to remove specific transitive dependencies

Unit Testing. JUnit

The Concept of Unit Testing

- Testing the smallest isolated code units.
- Focuses on individual functions or methods.
- Detects bugs early in development cycle.
- Ensures individual components work as expected.
- Forms foundation of the testing pyramid.

Core Characteristics of Unit Tests

- Tests must run completely independently.
- Execution happens quickly for rapid feedback.
- Results remain repeatable across all environments.
- Code isolates entirely from external systems.
- Automation allows frequent execution during builds.

Mock Objects

- Simulates behavior of real external dependencies.
- Isolates unit under test from subsystems.
- Replaces slow databases or network calls.
- Mimics complex scenarios or unexpected errors.
- Utilizes testing frameworks like Mockito effectively.

Integrating Unit Tests and Mocks

- Mocks enable true isolation for units.
- Tests focus strictly on local logic.
- Simplifies setup for complex test cases.
- Speeds up overall test suite execution.
- Improves code maintainability and test coverage.

`DateTime`

Timeline, Local, and Zone Time Overview

- Java 8 Update: Replaces java.util.Date and Calendar classes.
- Core Advantage: Core package classes are thread-safe.
- Immutability Principle: Objects cannot be modified after creation.
- LocalDate Class: Represents a date without any timezone.
- ZonedDateTime Class: Combines date, time, and timezone context.
- Instant Class: Represents a precise specific timeline point.
- ZoneId Utility: Handles geographic regions and time offsets.

Clock Operations and Date Calculations

- Clock Class: Accesses instants using a specific timezone.
- Instance Creation: Instantiate using the Clock.systemUTC() method.
- Chronological Comparison: Check date ordering via compareTo() method.
- Time Modification: Alter values using plus and minus methods.
- Duration Class: Represents precise time in seconds or nanoseconds.
- Static Factory Methods: Instantiate durations using ofHours() or parse().

Formatting and Parsing Dates and Times

- DateTimeFormatter Utility: Parses and prints date-time objects.
- Package Location: Resides within java.time.format package area.
- Safety Feature: Formatter class is completely thread-safe.
- Pattern Customization: Supports predefined and custom text patterns.
- Date Symbols: Uses Y for year and M for month.
- Time Symbols: Employs H for hours and m for minutes.
- Zone Symbols: Uses z for name and Z for offset.


JDBC

Overview of JDBC Architecture

- Full Name: Stands for Java Database Connectivity.
- Main Purpose: Connects Java applications to external databases.
- Core Layers: Comprises JDBC API and JDBC Drivers.
- Driver Manager: Registers all applicable database drivers.
- Driver Selection: Type 4 pure Java drivers are preferred.
- Automatic Discovery: Modern versions register drivers automatically.

Key JDBC Components

- Connection Interface: Establishes sessions with the database server.
- Statement Interface: Executes simple, non-parameterized SQL statements.
- PreparedStatement Interface: Handles parameterized SQL queries efficiently.
- CallableStatement Interface: Executes precompiled database stored procedures.
- ResultSet Interface: Holds data retrieved from selection queries.
- SQLException Class: Handles database application errors gracefully.

Queries with Statement Objects

- Object Creation: Generated using the connection's createStatement method.
- Command Container: Acts as a container for SQL commands.
- Running SELECTs: Executes data selections using the executeQuery method.
- Handling Results: Stores returned data rows inside a ResultSet.
- Running Modifiers: Executes modifications using the executeUpdate method.
- Resource Cleanup: Needs the close method to prevent leaks.

Working With Transactions Using JDBC

- Auto-commit mode is active by default.
- Disable it using setAutoCommit(false).
- This starts a manual transaction boundary.
- Call conn.commit() to save changes.
- Call conn.rollback() to cancel operations.
- Transactions ensure ACID-compliant execution.
- Isolation levels control concurrent data access.

Connection Pooling in JDBC

- Represents a cache of reusable connections.
- Active connections are shared among clients.
- Eliminates continuous connection creation overhead.
- Clients return connections for future reuse.
- Enhances application scalability and performance.
- Optimizes database system resource management.

Database and ResultSet Metadata

- Metadata describes database features and properties.
- Use DatabaseMetaData for DBMS capability information.
- It retrieves product names and versions.
- Use metaData.getDatabaseProductVersion() for engine details.
- Use ResultSetMetaData for query data structures.
- It exposes column names and types.


Thread
Synchronization

Introduction to Threads

- Operating systems allocate processor time in parts called quanta.
- Multithreading allows applications to perform tasks simultaneously.
- Each individual task is called a thread.
- Every process possesses its own set of variables.
- Multiple threads share the same application data.
- Shared data facilitates communication between different threads.

Thread Creation and Execution

- Threads can be created by extending the Thread class.
- Subclasses must override the core run method.
- Calling start executes the newly created thread.
- Implementing Runnable allows classes to extend other classes.
- The Runnable approach separates tasks from control code.
- The Callable interface supports asynchronous execution with results.
- Callable threads can safely throw exceptions during execution.

Thread Pools and Executors

- Thread pools manage sets of existing open threads.
- Tasks are held in queues when threads are busy.
- The Executor interface replaces low-level thread creation idioms.
- ExecutorService controls lifecycles with shutdown methods.
- The submit method handles Callable and Runnable tasks.
- ScheduledExecutorService manages delayed and periodic task execution.

Introduction to Synchronization

- Thread Communication Issues: Unsynchronized communication between multiple threads can produce critical errors like thread interference and memory consistency bugs.
- Core Purpose of Synchronization: Synchronization acts as a process that orders or divides access to shared resources, leading to the sequential execution of threads to prevent data corruption.
- Memory Ambiguity: To increase execution efficiency, the Java Runtime Environment (JRE) saves local copies of variables inside individual thread stack memories.
- Visibility Challenges: Changes made to a variable inside one thread's stack memory are not always visible to other threads immediately because JVM write/read timings to the main heap memory are unpredictable.
- The Volatile Solution: Using the volatile keyword guarantees that a variable is always read from and written directly to the main memory, ensuring immediate visibility across all threads without blocking execution.

Mechanisms for Thread Synchronization

- Blocking vs. Non-Blocking: Synchronization is categorized into non-blocking mechanisms (like volatile) and blocking mechanisms that restrict concurrent resource access.
- Monitor-Based Locking: Every object in Java possesses an implicit monitor that threads use to lock or unlock access; other threads attempting to enter a locked monitor are automatically blocked.
- Synchronized Methods and Blocks: Synchronized methods lock the entire object instance during execution, while synchronized blocks narrow the scope to critical code sections to allow non-critical code to run in PArallel.
- Advanced Synchronization APIs: The java.util.concurrent.locks framework provides highly sophisticated, professional-grade control over thread synchronization beyond standard keyword.
- Enhanced Lock Capabilities: Unlike basic synchronization, the explicit Lock interface supports advanced operations such as interruptible waiting, non-blocking lock attempts via tryLock(), and specific timeouts.
- ReentrantLock Fairness: The ReentrantLock class allows threads to safely re-acquire their own locks and introduces configurable fairness settings to prevent thread starvation.
- Optimizing with ReadWriteLocks: ReentrantReadWriteLock separates access into a shared read lock and an exclusive write lock, significantly boosting performance in applications where reading happens far more equently than writing.

Interthread Communication and Deadlocks

- Object Monitor Methods: Every Java object utilizes its implicit monitor to provide three fundamental communication methods: wait(), notify(), and notifyAll().
- Thread Suspension via wait(): Calling wait() forces the active thread to instantly release its monitor lock and enter a dormant Wait Set, where it consumes zero CPU cycles.
- Waking Threads: The notify() method wakes up a single random thread from the Wait Set, while notifyAll() wakes up all waiting threads.
- Lock Competition: Awoken threads do not resume execution immediately; they transition to a blocked state and must successfully compete to re-acquire the object's monitor lock.
- Synchronized Context Requirement: You must hold the object's lock to call these communication methods; attempting to invoke them outside a synchronized method or block throws an IllegalMonitorStateException.
- The Danger of Deadlocks: A deadlock occurs when two or more threads develop a circular dependency on a pair of synchronized objects, causing them to remain locked forever while waiting for each other.

Introduction to High-Level Concurrency APIs

- Built within the core java.util.concurrent package to simplify multithreading.
- Categorized into concurrent collections, blocking queues, synchronizers, executors, locks, and atomics.
- Executors standardize running and managing threads through thread pools and task submissions.
- Atomics maintain thread-safe programming for complex compound operations without external locking.
- Synchronizers define specific thread interactions like data exchange or waiting for states.

Key Concurrent Collections

- Designed to let multiple threads safely read and write data simultaneously.
- ConcurrentHashMap utilizes bucket-level locking and Compare-And-Swap operations instead of global locking.
- Reads in ConcurrentHashMap are typically lock-free, relying on volatile memory visibility.
- CopyOnWriteArrayList creates a completely fresh copy of the underlying array upon any mutation.
- Copy-on-write mechanics are highly optimized for scenarios where reads vastly outnumber writes.

Blocking Queues and Deques

- Tailored for implementing robust Producer-Consumer architectural patterns.
- BlockingQueue forces a retrieving thread to wait if the queue is currently empty.
- Storing operations are blocked automatically whenever the queue reaches full capacity.
- BlockingDeque permits thread-safe, blocking operations from both ends of the collection.
- SynchronousQueue functions with zero data storage capacity, directly pairing insert and delete actions.

Thread Control Methods

- sleep() suspends execution for a specified period in milliseconds or nanoseconds.
- It throws an InterruptedException if the sleeping thread is interrupted by another thread.
- join() forces the current running thread to wait until the target thread finishes its task.
- yield() allows a thread to temporarily step down and give up its time quantum to other threads.
- isAlive() is used as a diagnostic tool to check if a thread is currently operating.

Thread States and Interruption Handling

- Threads move through discrete lifecycle states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED.
- Interruption acts as a polite signal asking a thread to gracefully stop, rather than forcefully killing it.
- void interrupt() sends an interrupt request and flips the thread's internal status flag to true.
- static boolean interrupted() checks the current thread's flag and automatically clears it back to false.
- boolean isInterrupted() checks the flag of a target thread without modifying its status.

Semaphore and CyclicBarrier Classes

- Semaphore restricts and manages the total number of parallel threads accessing a shared resource.
- Threads call acquire() to request a permit or block until one becomes available.
- Threads must call release() inside a finally block to reliably return permits back to the pool.
- CyclicBarrier blocks a predefined number of threads until they all meet at a common sync barrier.
- Once the exact threshold of threads is met, the barrier opens and releases all waiting threads together.

CountDownLatch and Exchanger Classes

- CountDownLatch blocks threads until a specific set of external tasks or initialization events finish.
- Waiting worker threads call await() to pause execution until the internal countdown hit zero.
- Cooperating threads call countDown() to decrement the internal event counter by one.
- Exchanger establishes a simple, two-way data swap point directly between a pair of threads.
- The exchange() method passes a local object and blocks until the second thread provides its object.

Introduction to the Fork/Join Framework

- Added to the java.util.concurrent package in Java 7 as an implementation of the ExecutorService interface.
- Specifically designed to support parallel programming by automating the division and balancing of large computing tasks.
- Simplifies the process of creating, scaling, and managing worker threads across available system processors.
- Employs a divide-and-conquer strategy, recursively breaking tasks down until they become small enough to solve directly.

Core Hierarchy and Task Management

- ForkJoinPool manages worker threads and dynamically optimizes execution using a built-in work-stealing algorithm.
- ForkJoinTask<V> serves as the fundamental abstract base class for defining lightweight, splitable computing units.
- RecursiveAction extends ForkJoinTask for processing operations that do not return any value, operating similarly to a Runnable.
- RecursiveTask<V> extends ForkJoinTask for processing parallel operations that calculate and return a result, similarly to a Callable.

Essential Task Methods

- fork() submits a split subtask for asynchronous execution, allowing the parent thread to continue working without blocking.
- join() forces the calling thread to wait until a subtask finishes, returning its result to be combined into the main operation.
- compute() defines the core execution block where developer logic decides whether to process elements directly or split them.
- invoke() and invokeAll(...) bundle splitting and execution processes together to recursively implement divide-and-conquer logic.

Stream Parallelism (Implicit Concurrency)

- Java 8 streams execute sequentially by default unless explicitly triggered to distribute workloads among background cores.
- Appending .parallel() or executing .parallelStream() switches the framework into implicit parallelism mode.
- Parallel streams automatically break up internal data into individual RecursiveTask operations executed behind the scenes.
- Applications share a default global thread pool size, meaning competing parallel flows should have structured pool limits to prevent performance bottlenecks.


{ JSON }
< XML >

Working with JSON

- JSON is a lightweight data interchange format used primarily for transmitting data between servers and web applications.
- The structure relies on key-value pairs and supports primitives such as Strings, Numbers, Booleans, Arrays, and Null.
- Java applications can utilize libraries like Jackson or Gson to automate data serialization and deserialization instead of manually building strings.
- Processing lists of objects requires developers to explicitly specify the collection type when reading JSON arrays.
- Annotations like @SerializedName in Gson or @JsonProperty in Jackson allow customization when Java field names do not match JSON keys.

XML Fundamentals and Structure

- Extensible Markup Language (XML) is a text-based format designed to store, represent, and transfer arbitrary data structures.
- It serves as the foundation for widespread web technologies including XHTML, RSS, ATOM, and AJAX.
- A well-formed XML document must adhere to syntax rules including having a single root tag, proper element nesting, and quoted attribute values.
- Core components of an XML file include document declarations, elements or tags, attributes, comments, and unescaped Character Data (CDATA) sections.
- Main benefits include separating content from presentation and cross-platform support, though it can become inefficient with very large datasets.

XML Validation and Schemas

- Validation enforces specific business rules, ensures data integrity, and minimizes structural errors in XML documents.
- Document Type Definitions (DTDs) provide a basic validation method but use a different syntax and offer limited control over data types.
- XML Schema Definitions (XSD) are highly robust, written in XML syntax, and offer precise control over complex data structures.
- XSD building blocks consist of element declarations that define allowed tags and attribute declarations that define metadata.
- The standard integration workflow involves defining the XSD structure, creating well-formed XML content, and running validation via a DOM or SAX parser.

Streaming Architectures: SAX vs. StAX

- Tree-Based Processing: DOM (Document Object Model) loads the entire XML document into memory as a complete object tree structure.
- Streaming Processing: SAX and StAX parse XML sequentially, building or reading the data one individual node at a time.
- Navigation Capabilities: DOM supports bi-directional navigation and random access, while streaming processors only allow forward-only traversal.
- Data Modification: DOM supports full CRUD operations (Read, Write, and Modify), whereas SAX is strictly read-only.
- Memory and Speed: Streaming processors maintain a low, constant memory footprint and high speed, unlike DOM which consumes memory proportional to the file size.

Streaming Architectures: SAX vs. StAX

- Flow Control: The XML parser controls execution in push-streaming (SAX), whereas the application code drives execution in pull-streaming (StAX).
- Event vs. Cursor: SAX is entirely event-driven, while StAX relies on a developer-driven cursor or iterator mechanism.
- Callback System: Developers write specific handler methods in SAX, and the parser automatically pushes data into those callbacks.
- Active Pulling: StAX applications explicitly call methods like .next() to retrieve data only when the application is ready.
- Logic Flexibility: StAX offers greater flexibility, allowing the application to skip unneeded XML sections or halt the parsing loop midway.

Java Architecture for XML Binding (JAXB)

- Object-Binding Concept: JAXB maps XML schemas directly into strongly-typed Java domain classes (POJOs) rather than generic nodes.
- Unmarshalling Process: The unmarshaller is responsible for converting raw XML data back into accessible Java objects.
- Marshalling Process: Marshalling allows developers to convert existing Java object trees back into structured XML files.
- Required Metadata: Configuration requires specific source-code annotations, such as labeling root classes with @XmlRootElement.
- Element Mapping: The @XmlElement annotation explicitly binds a specific class variable to a corresponding XML element tag.
- Architectural Limitations: JAXB demands high memory to store the object tree and does not work natively in Android environments.

The latest release of the Java Development Kit is JDK 26 (specifically version 26.0.1, released on April 21, 2026).

- HTTP/3: Native support added directly to the standard HTTP Client.
- AOT Object Caching: Significantly speeds up JVM application startup and warmup.
- Faster G1 GC: Reduced synchronization bottlenecks for better performance.
- Final Fields: Warnings added for changing `final` fields via reflection.
- Applet API: Completely removed from the platform.
- Previews: Updates to lazy constants and primitive types in `switch` patterns.

QUIZ

- recall all that we have learned

1. Human's ability smell petrichor (smell of wet earth from rain) is greater than a _____'s ability to smell blood in water.

1. DOLPHIN
2. BARK
3. SHARK
4. MAN

2. Your brain alone burns around ____ to _____ calories each day

1. 400 to 500
2. 500 to 600
3. 600 to 700
4. 700 to 800

3. _____'s national animal is the unicorn

1. German
2. Scotland
3. Greek
4. Vietnam

4. Giraffes are __ times more likely to get hit by lightning than people

1. 10
2. 20
3. 30
4. 40

5. There were active _____ on the moon when dinosaurs were alive.

1. rivers
2. volcanoes
3. mountains
4. seas

6. He designed the logo for the Chupa Chups lollipop brand.

1. Leo DaVinci
2. Ivan Aivazovsky
3. Ichigo Kurosaki
4. Salvador Dalí

7. Uzbekistan is a combination of the Turkic words “uz” (self) and
“bek” (master) and the Persian suffix “-stan” (____???____). This
essentially translates as the “Land of the Free”

1. group
2. land
3. nation
4. people

8. _____ is one of only two countries in the world that is "doubly landlocked", meaning it is surrounded entirely by other landlocked countries. To reach an ocean or sea, travellers must cross at least two national borders. The only other country with this distinction is Liechtenstein.

1. Kazakhstan
2. India
3. USA
4. Uzbekistan

9. Lemons float and limes sink in _____ because of differences in density and rind structure.

1. oil
2. soda
3. water
4. sea

10. If you take all the DNA in your cells and stretch them out it can reach from the 🌍 to the 🌙 over _____ times.

1. 1 mln
2. 100 K
3. 10 K
4. 1 K

THANK YOU!!

and good luck!!!!